ECE 276A Project 1: Orientation Tracking

Aditya Wirayutha Kaya Simto
Department of Computer Science and Systems Engineering
Kobe University

I. Introduction

This project estimates the orientation of an IMU over time. The gyroscope gives angular velocity, which can be integrated into orientation, but sensor bias causes drift. The accelerometer gives the gravity direction when linear motion is small. Gravity can correct roll and pitch, but it cannot correct yaw. The main problem is therefore to combine both sensors without losing the useful information from either one.

The data contains eleven sequences. Nine include VICON ground truth and two are test sequences. Following the handout [1], I calibrate the raw IMU data, integrate the gyroscope to obtain an initial quaternion trajectory, and refine it with projected gradient descent. I then use the estimated orientations to project the camera images into an equirectangular panorama.

II. Problem Formulation

At time t, $\mathbf{q}_t \in \mathbb{H}_*$ describes the body orientation relative to the world. My quaternion functions store $\mathbf{q} = (q_w, \mathbf{q}_v)$ as $[q_w, q_x, q_y, q_z]$. Before passing a quaternion to SciPy in `main.py`, I reorder it as $[q_x, q_y, q_z, q_w]$ because SciPy uses the scalar last order by default [4]. The IMU gives angular velocity $\boldsymbol{\omega}_t$ and acceleration $\mathbf{a}_t$ in the body frame. Since the sampling interval changes slightly, I use the measured interval τ_t at every step.

Motion model. I assume $\boldsymbol{\omega}_t$ is constant during one interval:

$$\mathbf{q}_{t+1} = f(\mathbf{q}_t, \tau_t \boldsymbol{\omega}_t) := \mathbf{q}_t \circ \exp\left(\left[0, \tfrac{\tau_t \boldsymbol{\omega}_t}{2}\right]\right), \tag{1}$$

where $\circ$ is quaternion multiplication. The increment is on the right because $\boldsymbol{\omega}_t$ is measured in the body frame.

Observation model. When linear acceleration is small, the accelerometer measures gravity in the body frame:

$$[0, \mathbf{a}_t] = h(\mathbf{q}_t) := \mathbf{q}_t^{-1} \circ [0, 0, 0, g] \circ \mathbf{q}_t, \tag{2}$$

where $g = 1$ in gravity units. The same model is $h(\mathbf{q}) = R(\mathbf{q})^\top \mathbf{e}_z$, with $\mathbf{e}_z = [0, 0, 1]^\top$.

Cost. I estimate the trajectory by minimizing both model errors:

$$\begin{aligned} \mathbf{r}_t^m &= 2 \log\left(\mathbf{q}_{t+1}^{-1} \circ f(\mathbf{q}_t, \tau_t \boldsymbol{\omega}_t)\right), \\ \mathbf{r}_t^a &= [0, \mathbf{a}_t] - h(\mathbf{q}_t), \\ c(\mathbf{q}_{1:T}) &= \tfrac{1}{2} \sum_{t=0}^{T-1} \|\mathbf{r}_t^m\|_2^2 + \tfrac{1}{2} \sum_{t=0}^{T} \|\mathbf{r}_t^a\|_2^2. \end{aligned} \tag{3}$$

The first term follows the gyroscope motion. The second term follows gravity. The code includes $t = 0$ in the gravity term. Since $\mathbf{q}_0$ is fixed, this adds the same initial error to every candidate trajectory.

Constraint. Each quaternion must have unit norm:

$$\min_{\mathbf{q}_{1:T}} c(\mathbf{q}_{1:T}) \quad \text{s.t.} \quad \|\mathbf{q}_t\|_2 = 1 \ \forall t. \tag{4}$$

The constraint set is the unit sphere $S^3 \subset \mathbb{R}^4$. Its Euclidean projection is

$$\Pi_{\mathbb{H}_*}(\mathbf{q}) = \mathbf{q} / \|\mathbf{q}\|_2. \tag{5}$$

This projection is only quaternion normalization. I fix the initial orientation as $\mathbf{q}_0 = [1, 0, 0, 0]$. Quaternions are convenient here because rotation matrices need more constraints and Euler angles become singular at pitch $\pm 90°$.

III. Technical Approach

A. Orientation Tracking

1) Data Preprocessing: The IMU file is a $7 \times N$ array containing time, acceleration, and gyroscope data. In `find_static_end`, I call the NumPy `sliding_window_view` function [3] with 50 samples to find when the sensor starts moving:

$$n_s = \min\left\{k : \max_{d \in \{x,y,z\}} \operatorname{std}(G_{k:k+49,d}) > 1.5\right\}. \tag{6}$$

The starting time is different in each dataset, so one fixed value would not work well. Calling moving data stationary would make the calibration confidently wrong, which is not the confidence I need here.

Here S_a and S_g are the sensor sensitivities, and V_{ref} is the reference voltage. I compute the scales and biases as

$$\begin{aligned} s_a &= \frac{V_{\text{ref}}}{1023 S_a}, \quad s_g = \frac{V_{\text{ref}}}{1023 S_g} \frac{\pi}{180}, \\ \mathbf{b}_g &= \overline{\mathbf{G}}_{1:n_s}, \qquad \mathbf{b}_a = \overline{\mathbf{A}}_{1:n_s} - \begin{bmatrix} 0 & 0 & 1/s_a \end{bmatrix}^\top. \end{aligned} \tag{7}$$

I then calibrate each measurement using

$$\mathbf{a}_t = s_a(\mathbf{A}_t - \mathbf{b}_a), \qquad \boldsymbol{\omega}_t = s_g(\mathbf{G}_t - \mathbf{b}_g). \tag{8}$$

This converts raw counts into g and rad/s. I estimate a separate bias for each dataset.

The VICON and IMU timestamps do not match exactly. I use spherical linear interpolation to place the VICON rotations at the IMU times, and I remove invalid rotations. I check the calibration with

$$\frac{1}{|\mathcal{I}|} \sum_{t \in \mathcal{I}} \left\| \mathbf{a}_t - R_{\text{vicon}}(t)^\top \mathbf{e}_z \right\|_2 < 0.1 \text{ g}. \tag{9}$$

The residual is below the limit for every training dataset. The $+\mathbf{e}_z$ sign also gives much less error than $-\mathbf{e}_z$, so I use $+\mathbf{e}_z$. Thankfully, this sign choice was not a close election.

2) Optimization: I first integrate the gyroscope. One rotation step and the resulting dead reckoning trajectory are

$$\begin{aligned} E_t &= \exp\Big(\Big[0, \frac{\tau_t \boldsymbol{\omega}_t}{2}\Big]\Big), \\ \mathbf{q}_0^{\mathrm{dr}} &= [1,0,0,0], \\ \mathbf{q}_{t+1}^{\mathrm{dr}} &= \mathbf{q}_t^{\mathrm{dr}} \circ E_t. \end{aligned} \tag{10}$$

For a unit quaternion, $\|2\log \mathbf{q}\| = 2\arccos(q_w)$. I define $\mathbf{f}_t = \mathbf{q}_t \circ E_t$ and $s_t = \langle \mathbf{q}_{t+1}, \mathbf{f}_t \rangle$. This gives the simpler cost

$$c(\mathbf{q}) = 2\sum_t \arccos^2(s_t) + \frac{1}{2}\sum_t \|\mathbf{a}_t - h(\mathbf{q}_t)\|_2^2. \tag{11}$$

I clip s_t to $[-1,1]$ before `arccos` to avoid numerical problems. The mathematics already knows this range, but floating point arithmetic occasionally needs a reminder.

For $\theta_t = \arccos(s_t)$, define $\gamma_t = -4\theta_t / \sin\theta_t$. I add the motion gradient to both quaternions in each time step:

$$\begin{aligned} \mathbf{g}_{t+1} &\leftarrow \mathbf{g}_{t+1} + \gamma_t \mathbf{f}_t, \\ \mathbf{g}_t &\leftarrow \mathbf{g}_t + \gamma_t (\mathbf{q}_{t+1} \circ E_t^{-1}). \end{aligned} \tag{12}$$

Direct division is unstable when $\theta_t = 0$, so I use $\gamma_t = -4/\operatorname{sinc}(\theta_t/\pi)$ [2]. The gravity model and its Jacobian are

$$\begin{aligned} h(\mathbf{q}) &= \begin{bmatrix} 2(q_x q_z - q_w q_y) \\ 2(q_y q_z + q_w q_x) \\ q_w^2 - q_x^2 - q_y^2 + q_z^2 \end{bmatrix}, \\ J(\mathbf{q}) &= 2\begin{bmatrix} -q_y & q_z & -q_w & q_x \\ q_x & q_w & q_z & q_y \\ q_w & -q_x & -q_y & q_z \end{bmatrix}, \\ \mathbf{g}_t &\leftarrow \mathbf{g}_t - J(\mathbf{q}_t)^\top (\mathbf{a}_t - h(\mathbf{q}_t)). \end{aligned} \tag{13}$$

Each optimization step moves against the gradient and normalizes the result:

$$\begin{aligned} \mathbf{q}_t^{(k+1)} &= \frac{\mathbf{q}_t^{(k)} - \alpha \mathbf{g}_t}{\|\mathbf{q}_t^{(k)} - \alpha \mathbf{g}_t\|_2}, \\ \alpha &= 10^{-3}, \qquad k = 0, \ldots, 1499. \end{aligned} \tag{14}$$

I keep the first pose fixed by setting $\mathbf{g}_0 = 0$. I also compare the analytic gradient with finite differences. The cost decreases smoothly and is almost stable before the last iteration, as shown in Figure 3.

B. Panorama

1) Data Preprocessing: The camera looks along body x. Body y points left and body z points up. For pixel (i,j), I define

$$\begin{aligned} c_x &= \frac{W-1}{2}, \quad c_y = \frac{H-1}{2}, \\ u_j &= \frac{j - c_x}{W}, \quad v_i = \frac{i - c_y}{H}. \end{aligned} \tag{15}$$

Image columns increase to the right and rows increase downward. These directions are opposite to body y and z, so both angles have a minus sign:

$$\lambda_{ij} = -u_j \,\mathrm{FOV}_h, \qquad \varphi_{ij} = -v_i \,\mathrm{FOV}_v\,. \tag{16}$$

Here λ is the horizontal angle and φ is the vertical angle. I use $\mathrm{FOV}_h = 60^\circ$ and $\mathrm{FOV}_v = 45^\circ$, matching the constants in `panorama.py`. The corresponding body ray is

$$\mathbf{r}_{ij} = \begin{bmatrix} \cos\varphi_{ij}\cos\lambda_{ij} \\ \cos\varphi_{ij}\sin\lambda_{ij} \\ \sin\varphi_{ij} \end{bmatrix}, \qquad \|\mathbf{r}_{ij}\|_2 = 1. \tag{17}$$

At the image centre, $\lambda = \varphi = 0$ and $\mathbf{r} = [1,0,0]^\top$. For each camera frame, I use the latest IMU orientation available at that time:

$$k(c) = \max\{k : t_k^{\mathrm{IMU}} \le t_c^{\mathrm{cam}}\}. \tag{18}$$

This choice prevents the camera frame from using a future orientation.

2) Projection and Computational Optimization: I rotate each body ray into the world frame:

$$\mathbf{p}_{ijc} = R_{k(c)} \mathbf{r}_{ij} = \begin{bmatrix} p_x & p_y & p_z \end{bmatrix}^\top. \tag{19}$$

The world azimuth and elevation are

$$\theta = \operatorname{atan2}(p_y, p_x), \qquad \psi = \arcsin(p_z), \tag{20}$$

where $\theta \in (-\pi, \pi]$ and $\psi \in [-\pi/2, \pi/2]$. In the code, p_z is clipped to $[-1,1]$ before `arcsin`. For $H_p = 480$ and $W_p = 960$, I convert these angles to panorama column m and row n with the equirectangular projection [5]:

$$\begin{aligned} m &= \left\lfloor \frac{\theta + \pi}{2\pi}(W_p - 1) \right\rfloor, \\ n &= \left\lfloor \frac{\pi/2 - \psi}{\pi}(H_p - 1) \right\rfloor. \end{aligned} \tag{21}$$

The update is $P[n,m] \leftarrow I_c[i,j]$. When pixels overlap, I keep the most recent one. There is no numerical optimizer in this part. I compute all camera rays once, so each frame only needs one matrix multiplication. I ignore the camera offset because the scene is far away, and I use an assumed field of view. The code implements the equations directly with NumPy and does not call PROJ. Figure 1 shows four examples using values from dataset 1.

IV. Results

A. Orientation Tracking

Table I compares the estimates with VICON. After projected gradient descent, the mean roll error decreases from 11.94° to 1.62°, and the mean pitch error decreases from 7.09° to 1.48°. Roll improves in eight datasets and pitch improves in all nine. Yaw improves by only 1.93° because the accelerometer gives no yaw information. Figure 2 shows this same pattern.

The final roll and pitch errors are between 0.98° and 2.18°. Their correlation with the accelerometer calibration residual is $r = 0.903$, as shown in Figure 4. The complete orientation curves are included in Appendix I, Figures 8 to 10.

Dataset 3 is the only case where the roll error increases, from 1.26° to 1.77°. Its gyroscope estimate is already good, so accelerometer noise moves the result slightly away from it.

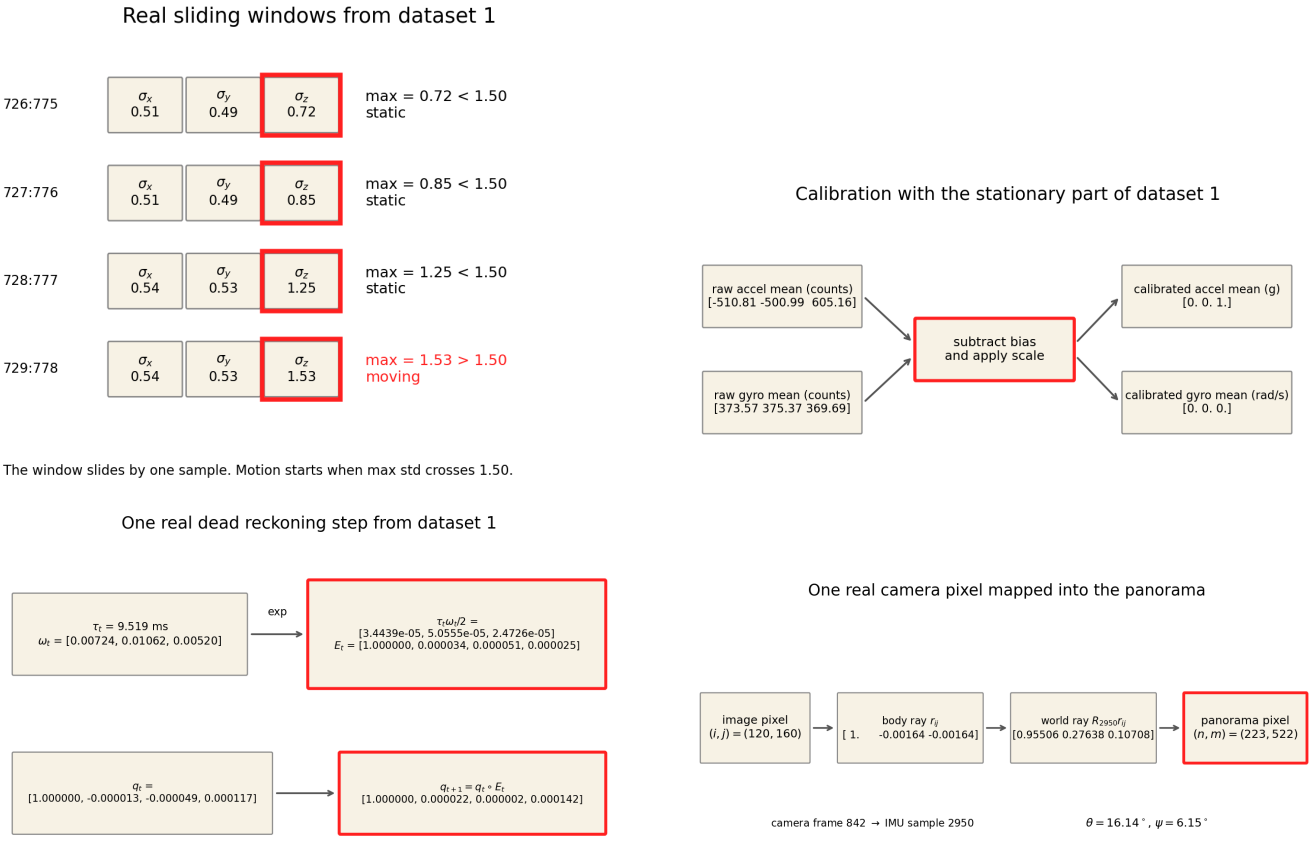


Fig. 1. Small numerical examples from dataset 1. The panels show static window detection, IMU calibration, one dead reckoning update, and one camera pixel projected into the panorama.

TABLE I

MEAN ABSOLUTE ERROR VS. VICON (DEGREES), GYROSCOPE ONLY → PGD.

ds	roll: gyro→PGD	pitch: gyro→PGD	yaw: gyro→PGD	geodesic: gyro→PGD
1	15.24 → **1.44**	8.10 → **1.01**	21.04 → 13.19	17.08 → 12.98
2	14.99 → **1.28**	14.41 → **1.08**	18.97 → 13.32	22.65 → 13.50
3	1.26 → 1.77	2.28 → **1.86**	14.20 → 14.46	14.64 → 14.82
4	15.87 → **2.18**	5.36 → **1.40**	38.90 → 39.04	42.25 → 39.16
5	18.57 → **1.90**	9.03 → **2.11**	18.71 → 16.74	27.46 → 17.22
6	8.27 → **1.83**	3.07 → **1.52**	4.68 → 4.68	11.15 → 5.90
7	12.72 → **2.08**	7.99 → **1.80**	6.28 → 5.02	15.57 → 5.83
8	3.19 → **1.10**	2.00 → **1.31**	10.47 → 10.11	11.34 → 10.49
9	17.38 → **0.98**	11.61 → **1.25**	29.61 → 28.89	36.46 → 29.09
mean	**11.94** → **1.62**	**7.09** → **1.48**	**18.10** → **16.16**	

Dataset 4 still has about 39° of yaw error, which is expected because gravity cannot correct yaw.

I also tried fitting new sensor scales from VICON. The accelerometer residual changed only from 0.0350 to 0.0339 g, while the gyroscope fit became unstable. Using the fitted scales increased the errors of datasets 3 and 9 to 60.06° and 69.25°. For this reason, I kept the datasheet values. The cost also decreases for test datasets 10 and 11, but their accuracy cannot be measured because VICON is not available. Their curves are shown in Figure 11.

B. Panorama

The estimated orientations produce recognizable room panoramas. Figure 5 compares VICON and PGD for dataset 1. The room layout is similar, but the whiteboard appears twice in the PGD result. This matches the remaining 13.19° yaw error. The room did not actually grow a second whiteboard. Panorama coverage is between 31.6% and 80.4%. More training examples are shown in Figure 6, and the test results are shown in Figure 7.

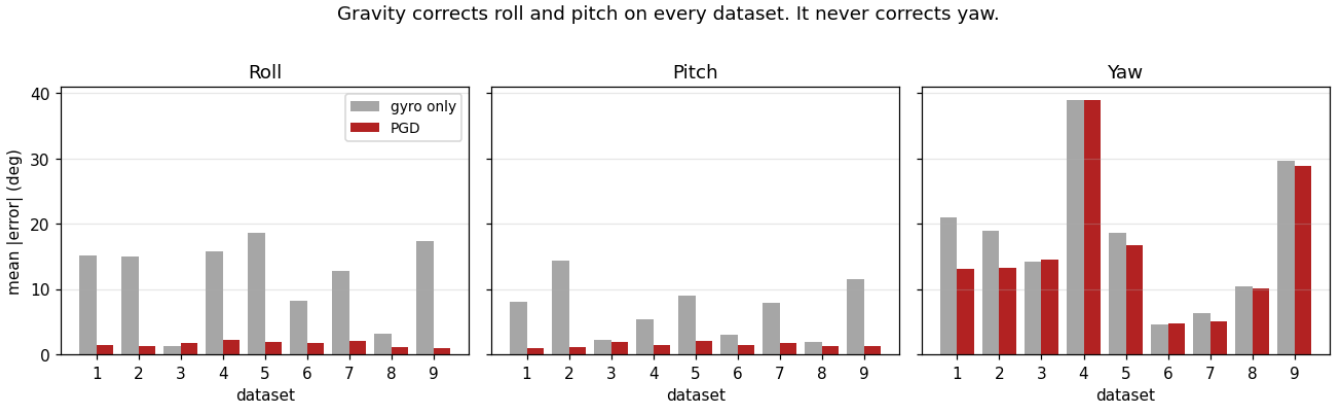


Fig. 2. Mean roll, pitch, and yaw error before and after projected gradient descent for all nine training datasets.

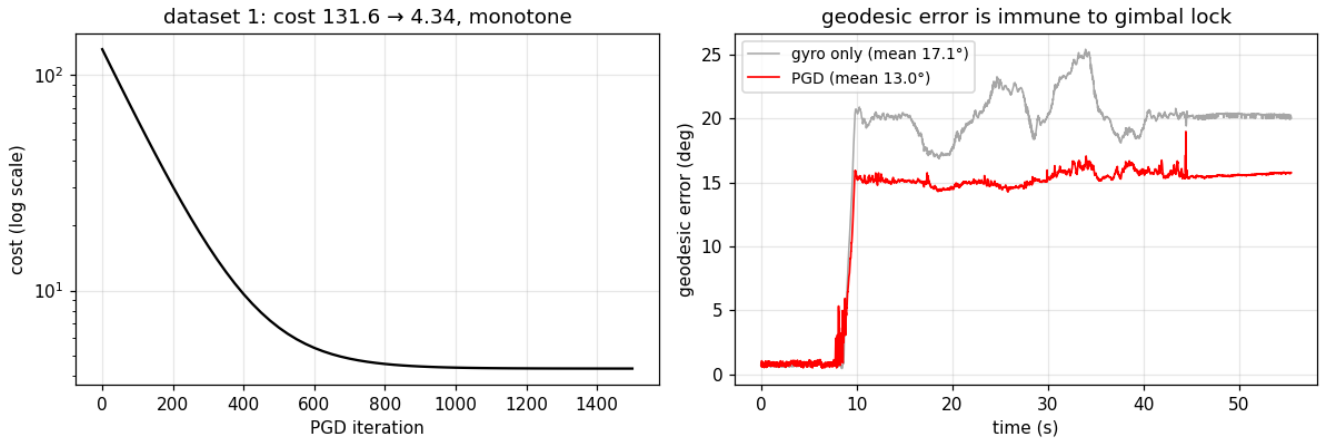


Fig. 3. Cost falls monotonically $131.6 \rightarrow 4.34$ (left); geodesic error vs. time (right).

V. Discussion

The main limitation is yaw. Rotating around gravity does not change the accelerometer measurement, so yaw depends only on the gyroscope and continues to drift. A magnetometer or visual reference would be needed to correct it. The method also uses the full trajectory, including future measurements. This helps the estimate, but it is not suitable for real time control.

Some settings are chosen manually. These include the camera field of view, the static threshold, and equal weights for both cost terms. The model also assumes that acceleration mainly comes from gravity. Fast linear motion breaks this assumption. Dataset 3 also suggests that using sensor noise to choose the cost weights may give a better result.

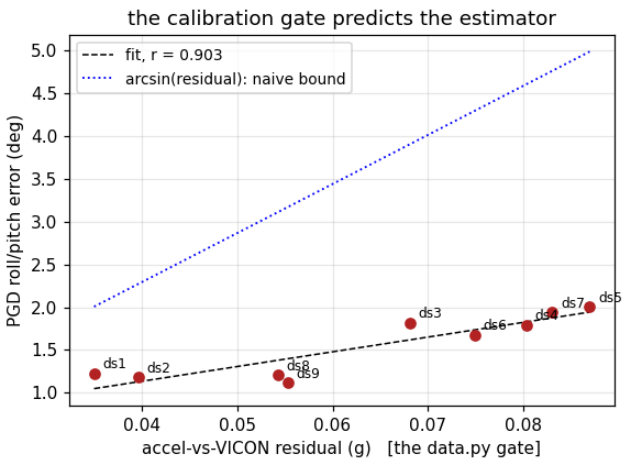


Fig. 4. Accelerometer calibration residual and final roll and pitch error for the nine training datasets.

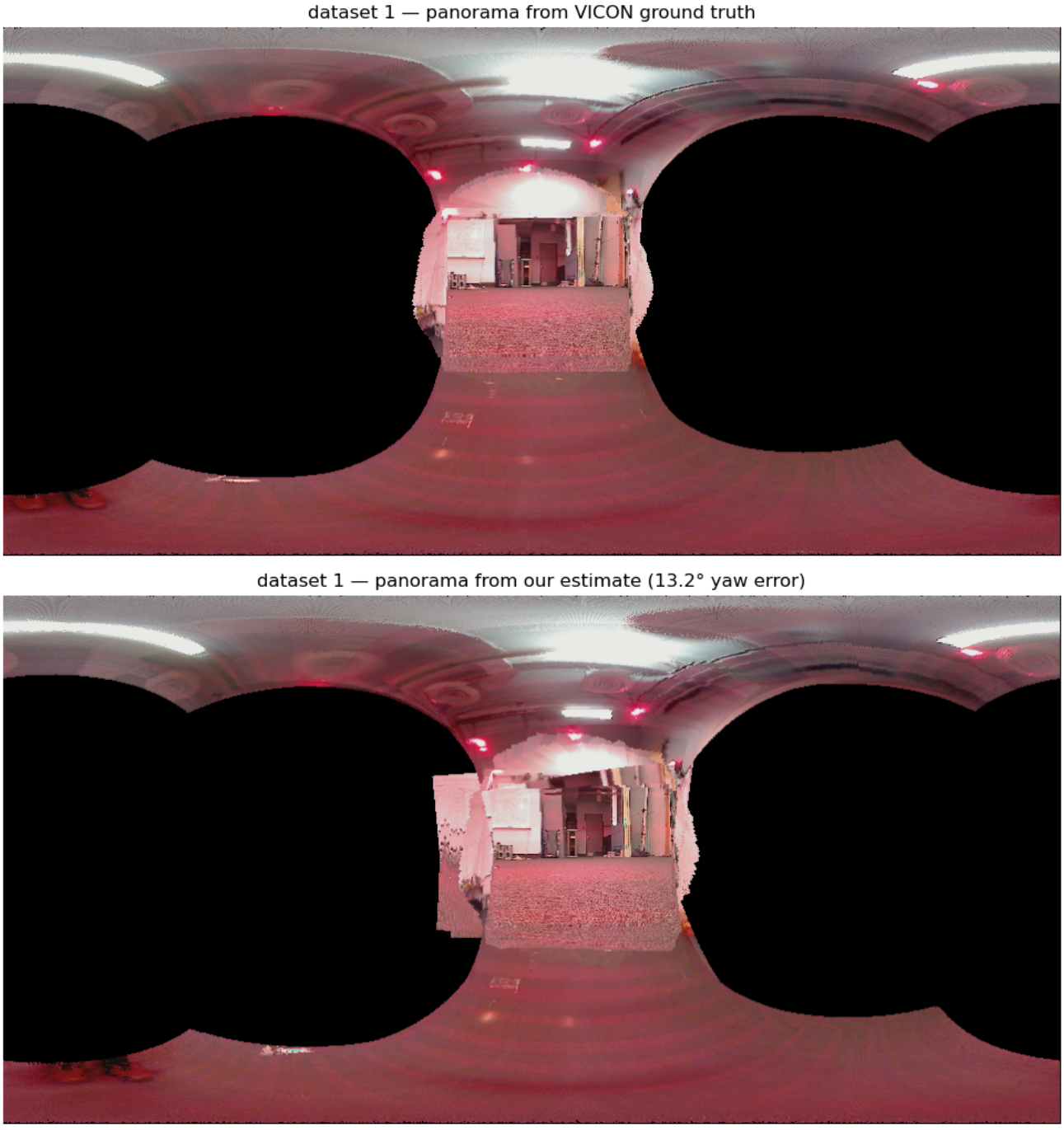


Fig. 5. VICON vs. estimated panorama; the doubled whiteboard is residual yaw.

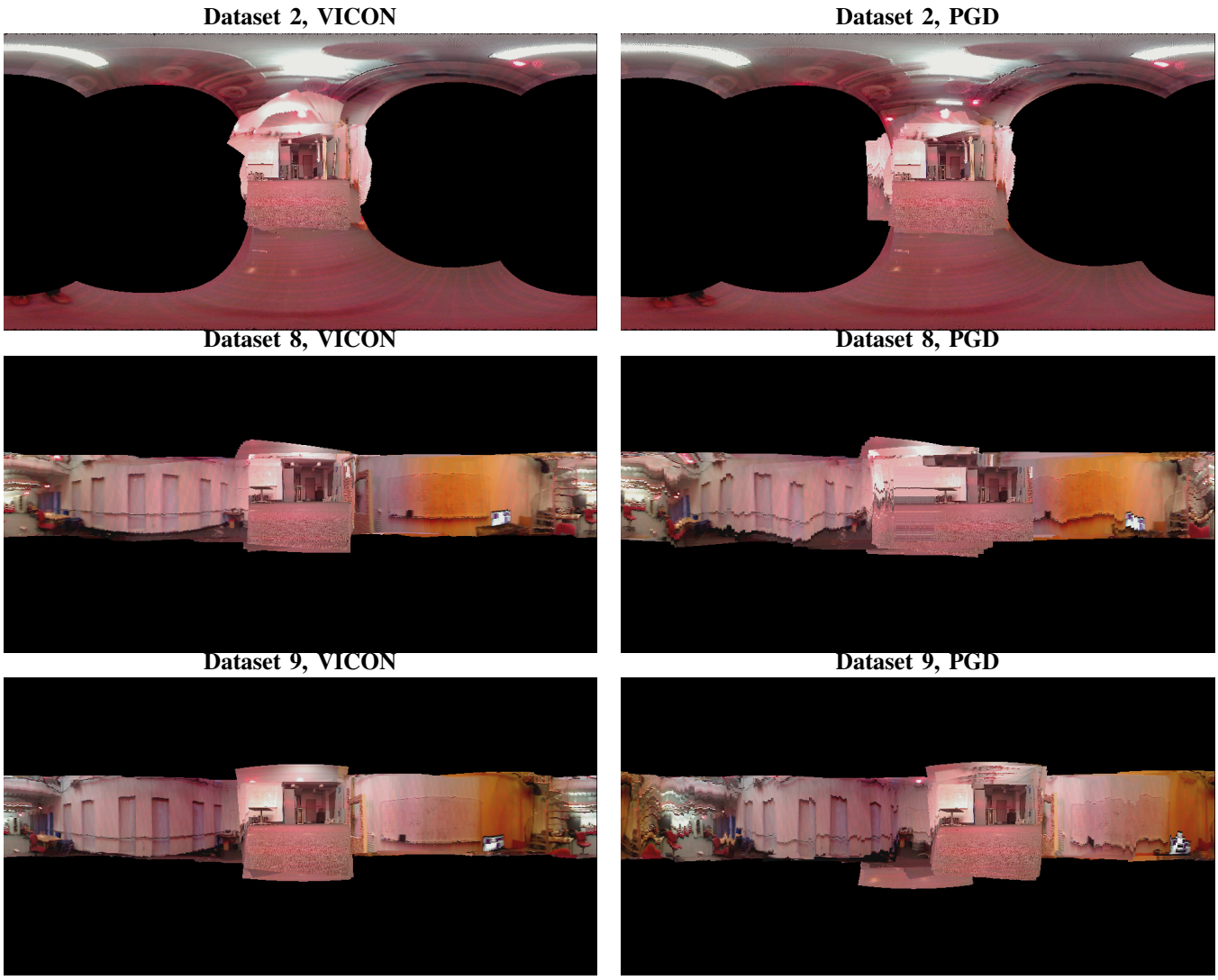


Fig. 6. Panoramas from VICON and projected gradient descent for training datasets 2, 8, and 9.

Dataset 10



Dataset 11

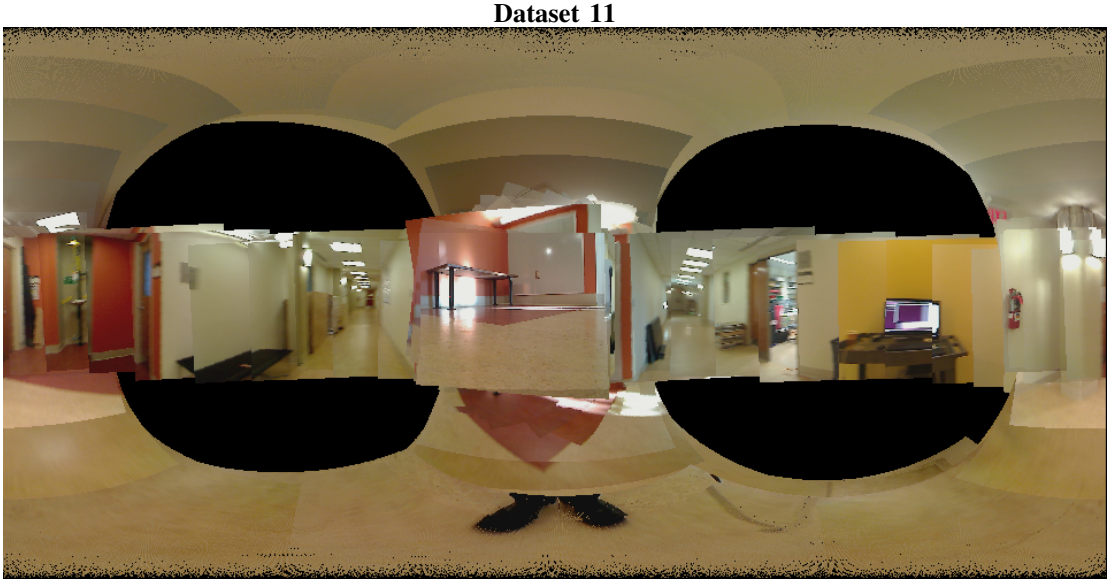


Fig. 7. Panoramas from projected gradient descent for test datasets 10 and 11. VICON is not available.

APPENDIX I
ORIENTATION CURVES FOR EACH DATASET

This appendix shows the roll, pitch, and yaw curves for every dataset. The main text only uses the summary plots.

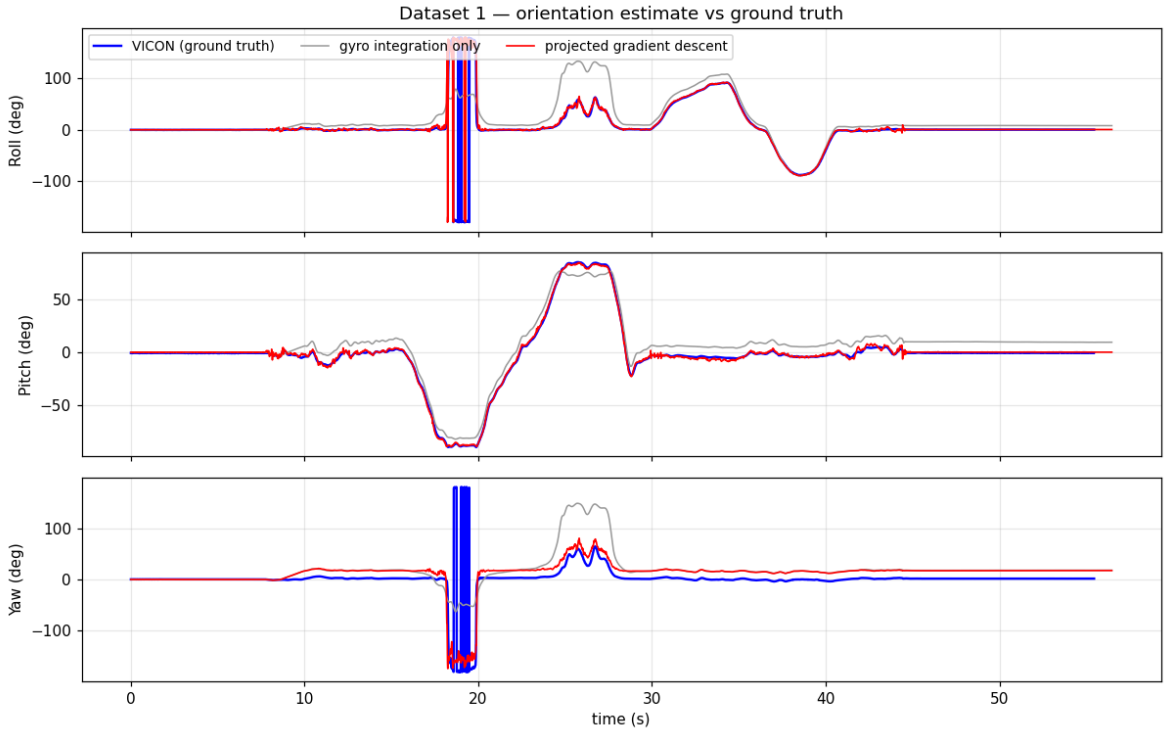


Fig. 8. Roll, pitch, and yaw for dataset 1. The plot compares VICON, gyroscope integration, and projected gradient descent.

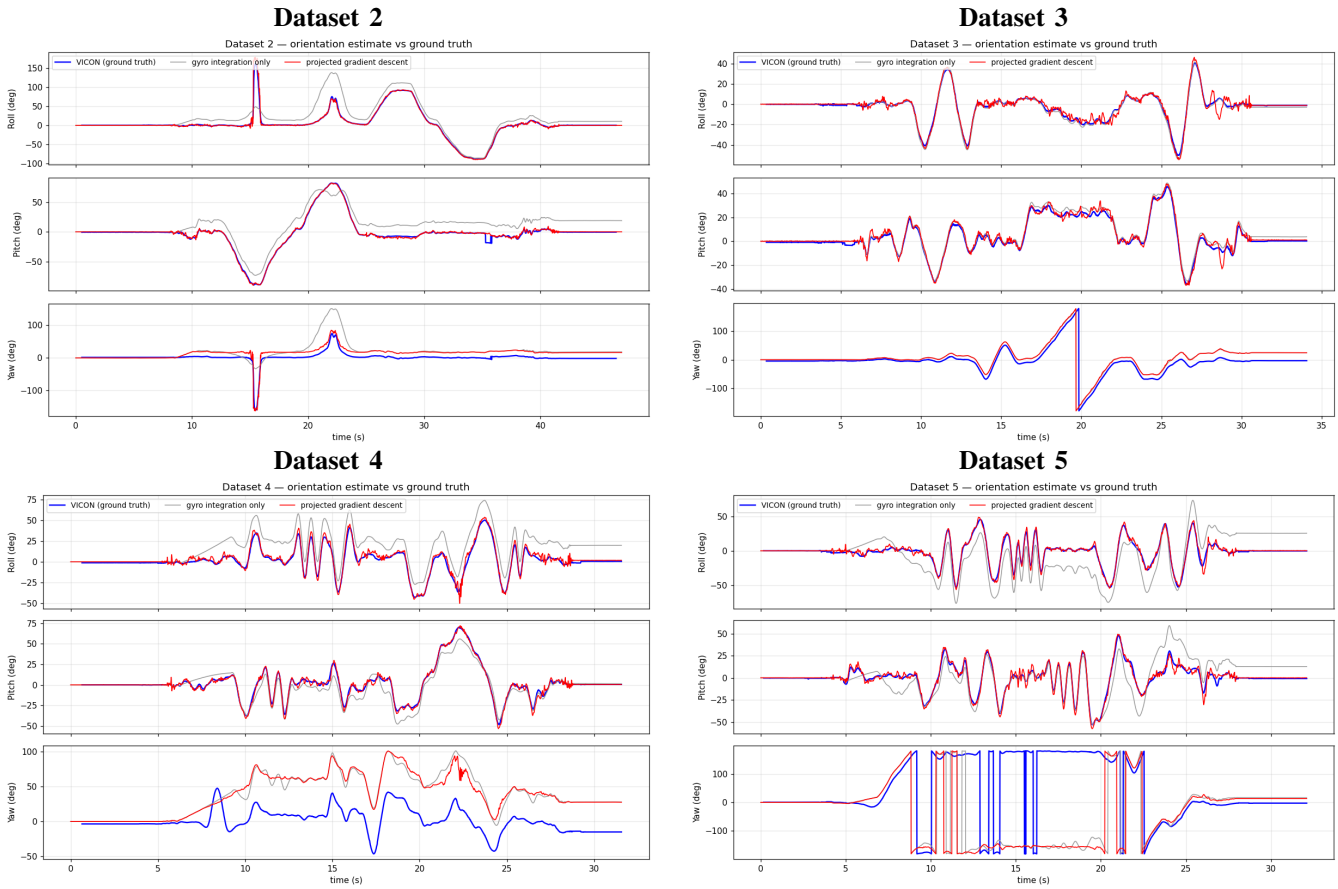


Fig. 9. Roll, pitch, and yaw for training datasets 2 to 5. Each plot compares VICON, gyroscope integration, and projected gradient descent.

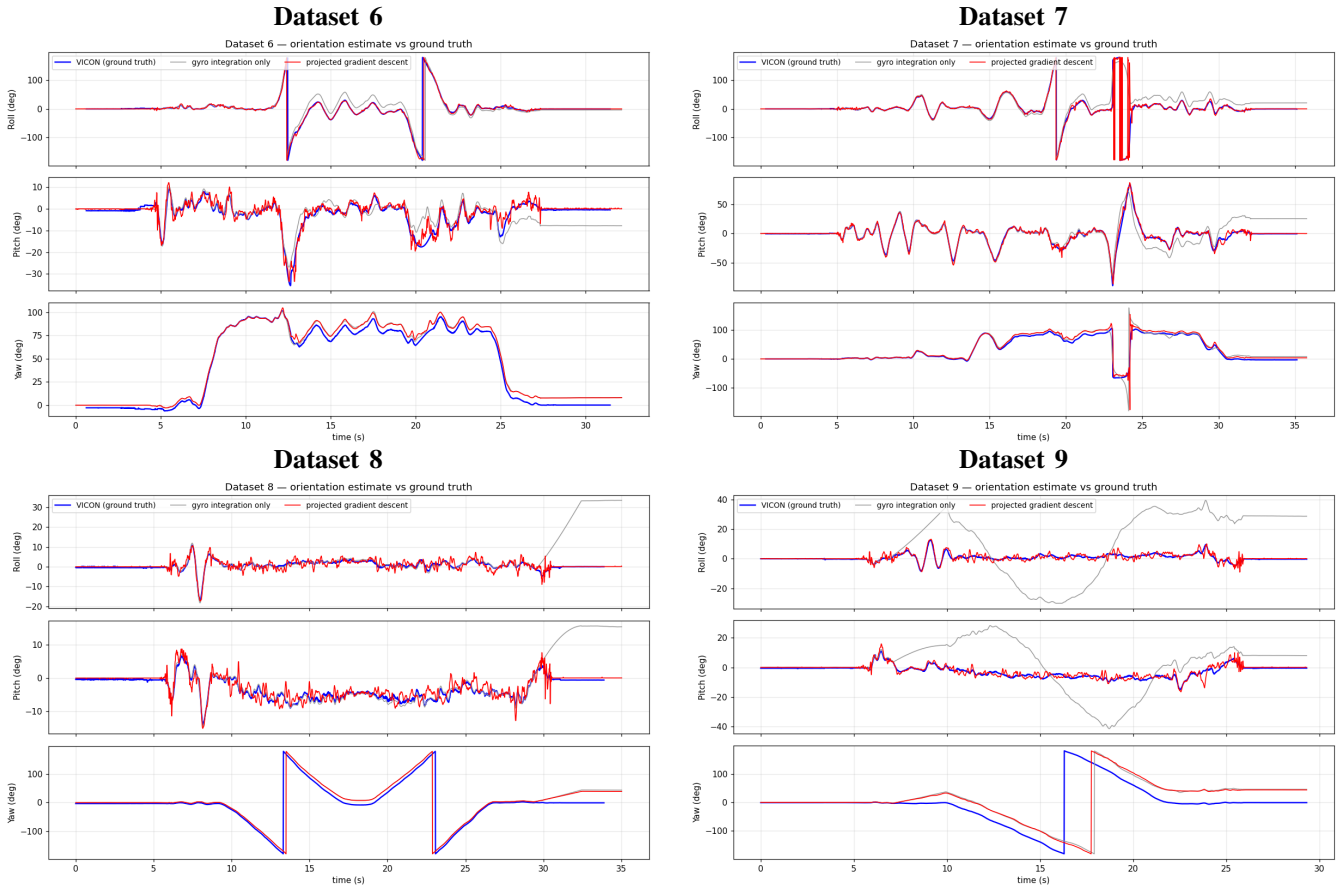


Fig. 10. Roll, pitch, and yaw for training datasets 6 to 9. Each plot compares VICON, gyroscope integration, and projected gradient descent.

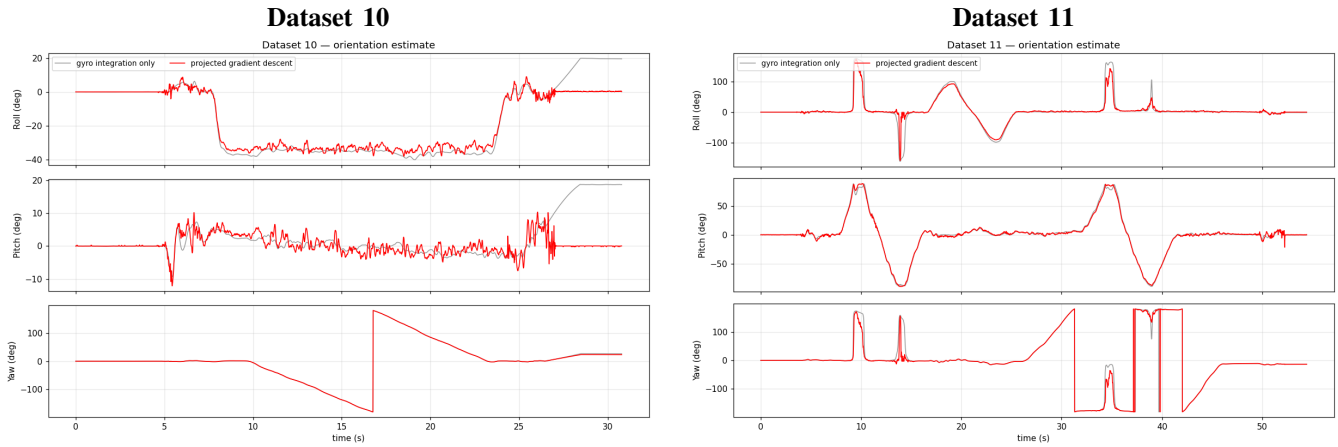


Fig. 11. Roll, pitch, and yaw from gyroscope integration and projected gradient descent for test datasets 10 and 11. VICON is not available.

REFERENCES

[1] N. Atanasov, *ECE 276A: Sensing & Estimation in Robotics*, Project 1 handout.

[2] NumPy Developers, "`numpy.sinc`." [Online]. Available: `https://numpy.org/doc/stable/reference/generated/numpy.sinc.html`

[3] NumPy Developers, "`numpy.lib.stride_tricks.sliding_window_view`." [Online]. Available: `https://numpy.org/doc/stable/reference/generated/numpy.lib.stride_tricks.sliding_window_view.html`

[4] SciPy Developers, "`Rotation.as_quat`." [Online]. Available: `https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.transform.Rotation.as_quat.html`

[5] PROJ Contributors, "Equidistant Cylindrical Projection." [Online]. Available: `https://proj.org/en/stable/operations/projections/eqc.html`